

文章笔记

Harness Design for Long-Running Application Development

基于 Anthropic Engineering Blog 整理

2026 年 3 月 24 日

文章作者: Prithvi Rajasekaran (Anthropic Labs)

发布日期: 2026 年 3 月 24 日

阅读时长: 阅读时长: 约 25 分钟

文章链接: <https://www.anthropic.com/engineering/harness-design-long-running-apps>

项目主页: <https://github.com/hqhq1025/ai-course-notes>

目录

1 引言	2
2 朴素实现的两个结构性困境	2
2.1 上下文窗口：连贯性退化与 Context Anxiety	2
2.2 自我评估偏差：Agent 的“自我感觉良好”	3
2.3 本章小结	4
3 前端设计：将主观审美转化为可评分标准	4
3.1 四维评分标准的设计	4
3.2 Generator-Evaluator 迭代闭环的实现	5
3.3 迭代过程中的关键发现	6
3.4 本章小结	7
4 全栈编码的三 Agent 架构	7
4.1 架构概览	7
4.2 Sprint Contract：弥合规格与实现的间隙	8
4.3 Evaluator 的调优：一个多轮人工过程	9
4.4 Retro Game Maker：Solo vs. Full Harness 对比	9
4.5 本章小结	10
5 Harness 的迭代简化	10
5.1 简化的核心原则	11
5.2 Opus 4.6 带来的范式转变	11
5.3 结构性简化：移除 Sprint 构造	12
5.4 DAW 构建实验：简化后的 Harness 实战	12
5.5 三个版本的综合对比	14
5.6 本章小结	14
6 总结与延伸	14
6.1 Harness 设计的演化论	14
6.2 三条实践建议	14
6.3 结构化反思	15
6.4 拓展阅读	15

1 引言

本文来自 Anthropic Labs 工程师 Prithvi Rajasekaran，是 Anthropic 关于 **harness design** (编排/脚手架设计) 系列文章的续篇。文章聚焦两个相互关联的前沿问题：如何让 Claude 产出**高质量的前端设计**（涉及主观审美判断），以及如何让 Claude 在**无人干预**的情况下完成完整应用的构建（涉及可验证的正确性与可用性）。

作者从 **GAN** (Generative Adversarial Networks, 生成对抗网络) 中汲取灵感，设计了 generator-evaluator 分离架构。在前端设计领域验证有效后，将其进一步扩展为 **planner-generator-evaluator** 三 agent 流水线，用于长时间自主编码任务。最终成果是在多小时的自主编码会话中产出功能完整的全栈应用——包括 Retro Game Maker 和浏览器内 DAW (Digital Audio Workstation)。

本文的核心贡献

1. 提出了将**主观设计质量**转化为可评分维度的方法论，使 LLM 的设计产出从“安全但平庸”跃升至具有创意风险的水平
2. 构建了 **planner-generator-evaluator** 三 agent 架构，显著提升长时间自主编码的产出质量
3. 展示了随着模型迭代 (Opus 4.5 → 4.6) 如何**系统性地简化 harness**：移除不再必要的组件、保留仍有价值的组件
4. 总结了 harness 设计的演化论——有趣的 harness 组合空间不会缩小，只会移动

文章的方法论意义超越了具体的应用场景：它展示了 AI 工程师如何在“模型能力”和“系统设计”之间找到动态平衡。

2 朴素实现的两个结构性困境

在深入 harness 设计之前，作者首先剖析了“单 agent 直接执行”模式在长时间任务上遇到的两个系统性失败模式。这些失败模式并非偶发——它们是模型架构和训练方式的结构性后果。

2.1 上下文窗口：连贯性退化与 Context Anxiety

模型在执行长时间任务时，随着 context window 逐渐填满，输出的连贯性会显著下降。这并不意外——更多的上下文意味着更多的注意力分散，更难保持对当前任务的聚焦。

但作者观察到一个更微妙的现象：**context anxiety** (上下文焦虑)。当模型“感知到”自己接近上下文长度限制时，会开始**提前草草收尾**——不是因为任务完成了，而是因为它“觉得”快没空间了。

Context Reset vs. Compaction: 两种应对策略

针对上下文膨胀，作者对比了两种策略：

Compaction (原地压缩)：

- 做法：对对话历史早期部分进行摘要压缩，同一个 agent 在缩短后的历史上继续工作
- 优点：保持了 agent 的连续性，不需要额外的编排逻辑
- 缺点：agent 仍然“记得”自己已经工作了很久，context anxiety 依然存在

Context Reset (完全重置)：

- 做法：彻底清空上下文，启动全新 agent，通过结构化的 handoff artifact 传递前一个 agent 的状态和下一步任务
- 优点：提供“干净的开始”，彻底消除 context anxiety
- 缺点：增加编排复杂度、token 开销和延迟；handoff artifact 必须足够完整

在早期测试中，**Claude Sonnet 4.5** 的 context anxiety 严重到 compaction 不足以应对，因此 context reset 成为 harness 设计的必要组件。

值得注意的是，context reset 引入了一个新的工程挑战：如何设计 handoff artifact 使得新 agent 能够无缝衔接前序工作。这个问题在后续的三 agent 架构中通过文件通信机制得到了系统性解决。

2.2 自我评估偏差：Agent 的“自我感觉良好”

第二个失败模式更加隐蔽且更难对付。当要求 agent 评估自己产出的工作时，它们表现出**系统性的正向偏差**。

Agent 自我评估的结构性缺陷

“When asked to evaluate work they’ve produced, agents tend to respond by confidently praising the work—even when, to a human observer, the quality is obviously mediocre.”

这个问题在**主观任务**（如前端设计）上尤为突出，因为不存在像单元测试那样的二值化验证手段。“一个布局是否感觉精致还是通用”是一个判断调用（judgment call），而 agent 在评判自己的工作时会可靠地偏向正面。

但即使在**有可验证结果的任务**上，agent 在执行过程中仍然会表现出不良判断——例如忽略明显的 bug 或对“差不多能用”的实现放行。

作者的关键洞察：**将生成和评估分离到不同的 agent**。分离本身并不能立刻消除评估的宽容倾向——evaluator 仍然是一个“倾向于对 LLM 产出宽容”的 LLM。但是，调优一个独立的 evaluator 使其保持怀疑态度，远比让 generator 对自身工作保持批判性更加**可行**（tractable）。一旦外部的严格反馈存在，generator 就有了具体的迭代目标。

这种“生成-评估分离”的思路直接来自 GAN 的核心思想，但在 LLM 系统中有其独特之处：GAN 中 discriminator 的训练是自动的，而 LLM evaluator 的校准需要大量人工介入。

2.3 本章小结

单 agent 的朴素实现面临两个结构性困境:(1)context window 膨胀导致的连贯性退化和 context anxiety, 使得长时间任务的后半段质量急剧下降;(2) 自我评估偏差导致 agent 对自身产出过度乐观, 无法自驱迭代改进。这两个问题共同构成了引入 multi-agent harness 的基础动机, 也解释了为什么仅靠 prompt engineering 无法解决长时间自主编码的核心挑战。

3 前端设计: 将主观审美转化为可评分标准

作者选择从前端设计任务入手来开发和验证 generator-evaluator 架构, 因为自我评估偏差在这里表现得最为突出。在没有任何干预的情况下, Claude 倾向于产出安全、可预测但视觉上毫无特色的布局——作者将之描述为 “technically functional but visually unremarkable”。

两个关键洞察驱动了 harness 设计:

1. 虽然审美不能完全量化 (individual tastes will always vary), 但可以通过编码设计原则和偏好来使其**可评分**。“Is this design beautiful?” 难以一致回答, 但 “does this follow our principles for good design?” 给了 Claude 具体的评分依据。
2. 通过将前端**生成**和前端**评分**分离, 可以创造驱动 generator 持续改进的反馈闭环。

3.1 四维评分标准的设计

作者定义了四个评分维度, 同时写入 generator 和 evaluator 的 prompt 中。这些维度的设计体现了对模型能力分布的深刻理解:

维度	定义	权重
Design Quality (设计质量)	设计是否感觉像一个连贯的整体 (coherent whole) 而非零件的拼凑? 颜色、排版、布局、图像和其他细节是否共同创造了独特的情绪 (mood) 和身份感 (identity)?	高
Originality (原创性)	是否有定制化的设计决策? 还是模板布局 (template layouts)、库默认值 (library defaults) 和 AI 生成模式 (AI-generated patterns) 的堆砌? 人类设计师应该能识别出刻意的创意选择。	高
Craft (工艺)	技术执行层面的检查: 排版层次 (typography hierarchy)、间距一致性 (spacing consistency)、色彩和谐 (color harmony)、对比度 (contrast ratios)。这是能力检查 (competence check) 而非创意检查。	低
Functionality (功能性)	独立于审美的可用性评估。用户能否理解界面功能、找到主要操作入口、完成任务而无需猜测?	低

表 1: 四维前端设计评分标准

权重分配的策略性考量

作者有意加大 Design Quality 和 Originality 的权重, 降低 Craft 和 Functionality 的权重。理由是 Claude 在技术执行 (Craft) 和功能实现 (Functionality) 方面已经表现不错——“required technical competence tended to come naturally to the model”。真正需要突破的是设计感和原创性, 在这两个维度上 Claude 常常产出高度通用的 “AI slop”。评分标准明确惩罚了典型的 AI 生成模式 (如 “purple gradients over white cards”——紫色渐变覆盖白色卡片), 并通过更高权重**推动模型承担更多审美风险** (aesthetic risk-taking)。

这一设计选择背后有一个更深层的洞察: 评分标准不仅是评估工具, **它们本身就是强 prompt 信号**。即使在第一轮迭代 (尚无 evaluator 反馈) 中, 仅仅因为 prompt 中包含了这些评分标准, generator 的产出就已经显著优于无任何 prompt 工程的 baseline。

3.2 Generator-Evaluator 迭代闭环的实现

整个系统基于 **Claude Agent SDK** 构建, 架构清晰:

1. **Generator** 根据用户 prompt 生成 HTML/CSS/JS 前端页面
2. **Evaluator** 通过 **Playwright MCP** 与**活页面** (live page) 直接交互——自主导航页面、截取屏幕截图、仔细研究实现细节——然后为每个维度打分并撰写详细评论 (detailed critique)
3. 评论反馈回 generator 作为下一轮迭代的输入

4. Generator 做出策略性决策：

- 分数趋势良好 → **Refine**：在当前方向上深化优化
- 当前方向无前途 → **Pivot**：转向完全不同的审美方向

5. 重复 5–15 轮迭代

Evaluator 使用了 **few-shot examples**（包含详细的分数拆解）进行校准，确保其判断与作者的偏好对齐，并减少跨迭代的分数漂移（score drift）。

由于 evaluator 每轮都在实际操作页面（而非评估静态截图），每个迭代周期耗时较长。完整运行可达 **4 小时**。

Playwright MCP 的关键角色

Playwright MCP 使得 evaluator 能够像真实用户一样与页面交互，而不仅仅是看一张截图。这意味着 evaluator 可以：

- 测试导航流程和交互响应
- 发现只有在实际操作中才能暴露的 bug
- 评估动态效果（动画、过渡、响应式行为）
- 从多个角度截图来全面评估视觉质量

这种“交互式评估”比“截图评估”提供了更丰富的反馈信号，但也显著增加了每轮迭代的时间成本。

3.3 迭代过程中的关键发现

通过大量实验运行，作者总结了几个重要观察：

1. 分数提升非线性。虽然 evaluator 的评分通常随迭代改善然后趋于平台期（plateau），但改善并非单调递增。作者“经常发现中间轮次的产出优于最终轮次”——这意味着 iteration 不等于 monotonic improvement。

2. 实现复杂度逐轮攀升。Generator 在 evaluator 的反馈推动下倾向于追求越来越 ambitious 的方案。这有利有弊——更 ambitious 意味着更多创意可能性，但也意味着更多潜在的实现缺陷。

3. 评分标准的措辞直接塑造产出。作者发现评分标准中的隐喻和形容词本身就是强信号。例如，包含“the best designs are museum quality”这样的短语会推动设计向特定视觉风格收敛。这揭示了一个重要的工程启示：**评分标准不仅是评估工具，它们同时是 prompt engineering 的一部分。**

Prompt 措辞的非预期效应

评分标准中使用的隐喻（如“museum quality”）会以作者未完全预期的方式引导 generator 的输出风格。这意味着在设计评分标准时，需要意识到每一个措辞选择都可能成为隐性的 style prompt。这既是一个工具（可以有意利用），也是一个风险（可能导致非预期的风格收敛）。

4. 创意跃迁的可能性。在一个令人瞩目的案例中，作者用“为一家荷兰艺术博物馆创建网站”作为 prompt。前 9 轮产出了一个干净的深色主题着陆页——视觉上精致但基本在预期之内。到第 10 轮，generator 彻底放弃了当前方向，将网站重新构想为一个**空间体验**（spatial experience）：

- 用 CSS perspective 渲染的棋盘格地板 3D 房间
- 艺术品以自由位置悬挂在墙上
- 通过门廊 (doorway) 导航切换画廊，而非传统的滚动或点击

这种从“精致的传统网页”到“3D 空间体验”的创意跃迁，是作者此前从未在单次生成中看到的。它展示了 evaluator-driven 迭代不仅能提升质量，还能在足够多的迭代中**催化质变**。

3.4 本章小结

前端设计实验验证了三个核心假设：(1) 主观审美可以通过编码设计原则转化为可评分维度；(2) generator-evaluator 分离能有效打破自我评估偏差，形成持续改进的闭环；(3) 评分标准本身就是强大的 prompt engineering 工具。将 Design Quality 和 Originality 的权重提高是关键设计决策——它将模型推向了其能力边缘，而这正是 evaluator 反馈最有价值的区域。

4 全栈编码的三 Agent 架构

有了前端设计实验的经验，作者将 GAN 启发的 generator-evaluator 模式扩展到全栈开发。代码审查 (code review) 和 QA 在软件开发生命周期中扮演着与 design evaluator 完全相同的结构性角色。

4.1 架构概览

在早期的 long-running harness 中，作者已经通过 initializer agent + coding agent + context reset 实现了连贯的多会话编码。但那个版本使用 Sonnet 4.5，context anxiety 严重到 context reset 必不可少。本次实验使用 **Opus 4.5**，其 context anxiety 问题显著减弱，因此作者选择**完全不使用 context reset**——所有 agent 在一个连续会话中运行，由 Claude Agent SDK 的自动 compaction 处理上下文增长。

Planner – Generator – Evaluator 三 Agent 架构

三个 agent 各自针对先前观察到的特定失败模式：

Planner（规划器）：将用户的 1–4 句简短 prompt 扩展为完整的产品规格（product spec）。关键设计决策：

- 被指示保持**高层次**的产品上下文和技术设计，而非详细的技术实现细节
- 如果 planner 在技术细节上犯错，错误会**级联**到下游实现
- 被要求在规格中融入 **AI 功能**的机会
- 被要求对**范围保持雄心**（be ambitious about scope）

Generator（生成器）：以 sprint 为单位工作，每次从规格中取一个 feature 实现。技术栈：

- 前端：React + Vite
- 后端：FastAPI + SQLite（后迁移至 PostgreSQL）
- 版本控制：git
- 每个 sprint 结束时进行自我评估，然后交给 QA

Evaluator（评估器 / QA）：通过 Playwright MCP 像真实用户一样操作运行中的应用，测试 UI 功能、API 端点和数据库状态。按维度打分（product depth, functionality, visual design, code quality），任一维度低于硬阈值则 sprint **失败**，generator 收到详细反馈。

4.2 Sprint Contract：弥合规格与实现的间隙

产品规格被有意保持在高层次（避免 planner 的细节错误级联到实现中）。但这引入了一个新问题：高层次的用户故事与具体的可测试实现之间存在间隙。为此，作者引入了 **sprint contract**（冲刺合同）机制。

Sprint Contract 的工作流程

在每个 sprint 开始前，generator 和 evaluator 进行一轮**谈判**：

1. **Generator 提案**：描述本 sprint 将构建什么，以及如何验证成功
2. **Evaluator 审查**：确认 generator 构建的是“正确的东西”（the right thing）
3. **双方迭代**：直到就“完成”的定义达成一致
4. **Generator 实现**：按合同编码
5. **Evaluator 验收**：按合同测试

Agent 之间的所有通信**完全通过文件**进行：一个 agent 写文件，另一个读文件并通过修改原文件或创建新文件来回应。这种设计简单且可追溯。

Sprint contract 的粒度可以非常细——例如在 Retro Game Maker 实验中，仅 Sprint 3（关卡编辑器）就有 **27 条**验收标准。

4.3 Evaluator 的调优：一个多轮人工过程

开箱即用的 Claude 是一个糟糕的 QA Agent

“Out of the box, Claude is a poor QA agent.”

在早期运行中，作者观察到 evaluator 的两个典型问题：

1. **宽容偏差**：evaluator 会识别出合法问题，然后说服自己这些问题“不是大问题”，最终批准工作。它在发现 bug 和判断 bug 严重性之间存在断裂。
 2. **表面测试**：evaluator 倾向于只测试“happy path”（正常路径），而非深入探查边缘情况（edge cases），导致更微妙的 bug 逃逸。
- 这两个问题意味着 evaluator 的校准不是一次性配置，而是需要**多轮人工审查**的调优过程。

作者的调优方法论是一个人工反馈闭环：

1. 阅读 evaluator 的日志
2. 找到 evaluator 的判断与作者自身判断分歧的案例
3. 更新 QA prompt 以解决这些分歧
4. 重新运行并重复上述步骤

经过多轮调优后，evaluator 能够发现相当具体和深入的问题。

4.4 Retro Game Maker：Solo vs. Full Harness 对比

作者使用相同的 prompt 分别在单 agent（solo）和完整 harness 上运行：

“Create a 2D retro game maker with features including a level editor, sprite editor, entity behaviors, and a playable test mode.”

模式	时长	成本	核心结论
Solo agent	20 min	\$9	核心功能（游戏运行）不可用
Full harness	6 hr	\$200	核心功能可用，有粗糙之处

表 2: Solo agent vs. Full harness 对比（Opus 4.5）

Harness 的成本是 solo 的 **20 倍以上**，但产出质量存在质的差异。

Solo agent 的问题：

- 初看界面似乎符合预期，但深入使用后问题大量涌现
- 布局浪费空间：固定高度的面板让大部分视口空白
- 工作流程僵硬：试图构建关卡时需要先创建精灵和实体，但 UI 没有任何引导
- **最致命的问题**：游戏本身**无法运行**——实体出现在屏幕上但不响应输入，entity 定义和 game runtime 之间的接线断裂（broken wiring），且没有任何表面指示告诉用户问题出在哪里

Full harness 的产出：

- Planner 从一句 prompt 扩展为 **16 个 feature、10 个 sprint** 的完整规格
- 远超 solo 的尝试范围：精灵动画系统、行为模板、音效音乐、AI 辅助精灵生成、关卡设计师、游戏导出与分享
- Planner 甚至读取了 Anthropic 内部的 frontend design skill 并据此创建了视觉设计语言
- Canvas 充分利用视口、面板尺寸合理、界面有一致的视觉身份
- 精灵编辑器更丰富：更干净的工具调色板、更好的颜色选择器、更可用的缩放控件
- 内置 Claude 集成，可以通过 prompt 生成游戏的不同部分
- **最关键：游戏可以玩**——玩家能移动角色，虽然物理引擎有粗糙之处（跳上平台后与之重叠），但核心循环运转

以下是 evaluator 在验收过程中发现的具体 bug 示例：

合同标准	Evaluator 发现
矩形填充工具允许拖拽填充区域	FAIL — 工具仅在拖拽起止点放置 tile，而非填充整个区域。 <code>fillRectangle</code> 函数存在但在 <code>mouseUp</code> 时未正确触发。
用户可选择并删除实体生成点	FAIL — <code>LevelEditor.tsx:892</code> 的 Delete 键处理要求 <code>selection</code> 和 <code>selectedEntityId</code> 同时设置，但点击实体只设置后者。
用户可通过 API 重排动画帧	FAIL — <code>PUT /frames/reorder</code> 路由定义在 <code>/frame_id</code> 之后，FastAPI 将 “reorder” 匹配为整数 <code>frame_id</code> 并返回 422 错误。

表 3: Evaluator 发现的具体 bug 示例 (Retro Game Maker)

Evaluator 的 Bug 报告质量

注意上表中 evaluator 的发现不仅指出了**什么失败了**，还精确定位了**为什么失败**——包括具体的文件名、行号、函数名和修复建议。这种水平的 bug 报告是经过多轮人工校准后才达到的。开箱即用的 LLM 会倾向于给出模糊的 “seems to not work properly” 式反馈。

4.5 本章小结

三 agent 架构通过角色分离系统性地解决了不同层面的问题：Planner 将简短 prompt 扩展为完整规格，解决了范围不足的问题；Generator 以 sprint 为单位聚焦实现；Evaluator 通过 Playwright MCP 进行真实用户视角的端到端测试，确保质量。Sprint contract 是连接高层规格与可测试实现的关键桥梁。虽然成本增加了 20 倍以上，但在“核心功能是否可用”这一最基本的维度上实现了质的飞跃——从“游戏无法运行”到“可以实际游玩”。

5 Harness 的迭代简化

第一版 harness 效果显著，但也笨重 (bulky)、缓慢 (slow)、昂贵 (expensive)。作者的下一步目标是在**不降低产出质量**的前提下简化 harness。

5.1 简化的核心原则

Harness 组件即模型能力假设

“Every component in a harness encodes an assumption about what the model can’t do on its own, and those assumptions are worth stress testing, both because they may be incorrect, and because they can quickly go stale as models improve.”

Harness 中的每个组件都编码了一个假设：“模型不能独立完成 X”。这些假设需要持续验证，原因有二：

- 1. 假设本身可能从一开始就不正确
- 2. 随着模型迭代，假设可能迅速过时

来自 Anthropic “Building Effective Agents” 博客的指导原则：“Find the simplest solution possible, and only increase complexity when needed.”

作者在简化方面的经验教训也很有价值。他首先尝试了激进的简化 (radical cutback) 并加入了一些创新想法，但未能复现原有 harness 的性能。更糟的是，激进简化使得难以判断哪些组件真正承载了性能 (load-bearing)。

基于这一经验，作者转向了更系统性的方法：每次只移除一个组件，观察其对最终结果的影响。这本质上是一种控制变量的消融实验 (ablation study) 方法。

5.2 Opus 4.6 带来的范式转变

在迭代过程中，Opus 4.6 发布。其关键能力改进直接对应了 harness 之前需要补偿的缺陷：

Opus 4.6 的改进	对 Harness 的影响
更细致的规划能力	Planner 的部分职责可由模型原生承担
更持久的 agentic task 执行	Sprint 分解可能不再必要
更可靠的大型代码库操作	Context reset 的必要性降低
更好的代码审查和调试能力	自我纠错能力增强, Evaluator 的边际价值变化
显著改善的 long-context retrieval	Compaction 和 context reset 的必要性降低

表 4: Opus 4.6 能力改进与 Harness 组件的对应关系

关键变化：Opus 4.5 上严重的 context anxiety 在 Opus 4.6 上基本消失。这意味着之前为了应对 context anxiety 而引入的 context reset 机制不再必要——agent 可以在一个连续会话中通过自动 compaction 处理上下文增长。

5.3 结构性简化：移除 Sprint 构造

作者首先移除了 **sprint 构造**。Sprint 结构的原始目的是将复杂工作分解为模型可以连贯处理的小块。Opus 4.6 的能力提升使得模型可能**无需这种分解**就能原生处理整个任务。

移除 sprint 后的关键决策：

Planner 保留。理由很实际：没有 planner，generator 会欠缺范围界定（under-scoped），直接从原始 prompt 开始构建，最终产出功能较少的应用。Planner 在将一句话 prompt 扩展为完整产品规格方面持续提供明确价值。

Evaluator 调整为末尾单次评估。从每个 sprint 评分改为整体运行结束后的单次（或少数几次）QA 通过。

Evaluator 的价值是任务依赖的，不是固定开关

“The practical implication is that the evaluator is not a fixed yes-or-no decision. It is worth the cost when the task sits beyond what the current model does reliably solo.”

在 Opus 4.5 上，构建任务处于 generator 能力的**边缘**，evaluator 在几乎所有 sprint 上都能发现有意义的问题。在 Opus 4.6 上，模型能力边界**外移**，许多之前需要 evaluator 把关的任务现在 generator 已能独立处理好。

但对于那些**仍处于 generator 能力边缘**的任务部分，evaluator 依然提供显著提升。这意味着：evaluator 的投入产出比随着模型改进而**动态变化**，harness 设计者需要持续重新评估。

此外，作者还改进了 harness 中 AI 功能构建的 prompting——具体来说，引导 generator 为每个应用构建一个 **proper agent**，能通过 tools 驱动应用本身的功能。这需要大量调优，因为相关知识足够新，Claude 的训练数据中覆盖较薄。

5.4 DAW 构建实验：简化后的 Harness 实战

使用更新后的 harness（Opus 4.6 + 无 sprint + 末尾 QA），作者测试了一个更具挑战性的 prompt：

“Build a fully featured DAW in the browser using the Web Audio API.”

Agent / 阶段	时长	成本
Planner	4.7 min	\$0.46
Build (Round 1)	2 hr 7 min	\$71.08
QA (Round 1)	8.8 min	\$3.24
Build (Round 2)	1 hr 2 min	\$36.89
QA (Round 2)	6.8 min	\$3.09
Build (Round 3)	10.9 min	\$5.88
QA (Round 3)	9.6 min	\$4.06
Total	3 hr 50 min	\$124.70

表 5: DAW 构建各阶段时长与成本 (Updated Harness + Opus 4.6)

关键观察：

1. 连贯运行超过 2 小时。 Builder 在**没有 sprint 分解**的情况下连贯运行超过 2 小时——这在 Opus 4.5 上是不可能的。这直接验证了移除 sprint 构造的决策。

2. QA 仍然发现真实缺陷。即使模型能力大幅提升，evaluator 在末尾 QA 中仍然抓到了有意义的问题：

第一轮 QA 反馈指出：

“Several core DAW features are display-only without interactive depth: clips can’t be dragged/moved on the timeline, there are no instrument UI panels (synth knobs, drum pads), and no visual effect editors (EQ curves, compressor meters). These aren’t edge cases—they’re the core interactions that make a DAW usable.”

第二轮 QA 继续发现：

- 音频录制仍为 stub（按钮可以切换但无实际麦克风捕获）
- Clip 的边缘拖拽缩放和分割未实现
- 效果可视化仅为数字滑块，而非图形化（无 EQ 曲线）

3. 最终产出可用。最终的浏览器 DAW 拥有完整的核心组件：arrangement view、mixer 和 transport。用户可以**完全通过 prompt** 与内置 AI agent 交互来创作歌曲——agent 能设置 tempo 和 key、铺设旋律、构建鼓轨、调整混音电平、添加混响效果。

4. 局限性清晰可见。作者坦诚指出：这远非专业的音乐制作程序，AI agent 的歌曲创作能力还有很大提升空间。此外，Claude 实际上无法“听到”音频，这使得 QA 反馈循环在音乐品味方面的有效性受限。

5.5 三个版本的综合对比

Harness 版本	时长	成本	特点
Solo agent (4.5)	20 min	\$9	Retro Game Maker 核心功能不可用
Full harness V1 (4.5)	6 hr	\$200	核心功能可用，有粗糙之处
Updated harness V2 (4.6)	3 hr 50 min	\$125	更简洁架构，同等或更高质量

表 6: 三个 Harness 版本的综合对比

从 V1 到 V2 的演进路径清晰：模型能力提升使得架构可以**同时降低复杂度和成本**，而不牺牲产出质量。这不是“用更多钱买更好的结果”，而是“用更少的脚手架获得同等或更好的结果”。

5.6 本章小结

Harness 简化的核心方法论是：系统性地逐个移除组件并观察影响，而非激进地一次性简化。从 Opus 4.5 到 4.6 的模型升级使得 sprint 结构可以被安全移除，evaluator 从每 sprint 评估改为末尾少次评估。成本从 \$200/6hr 降至 \$125/4hr，同时保持或提升了产出质量。关键洞察是 evaluator 的价值不是固定的——它随任务与模型能力边界的关系而动态变化。

6 总结与延伸

6.1 Harness 设计的演化论

核心论断：空间在移动，而非缩小

“The space of interesting harness combinations doesn’t shrink as models improve. Instead, it moves, and the interesting work for AI engineers is to keep finding the next novel combination.”

有趣的 harness 组合空间不会随着模型改进而缩小——它会**移动**。随着模型变强，某些 harness 组件变得不再必要（如 Opus 4.6 下的 sprint 分解），但新的可能性也随之打开（如让 generator 构建嵌入 AI agent 的应用）。AI 工程师的核心工作不是被动等待模型进步使 harness 过时，而是**主动寻找下一个有价值的组合**。

这一论断具有重要的方法论意义。它意味着 harness design 不是一个“最终会被模型进步淘汰”的过渡性工程领域，而是一个与模型能力**共同演进**的持久性工程领域。

6.2 三条实践建议

作者从这项工作中提炼了三条可携带的实践建议：

- 1. 实验驱动，阅读 trace。**用你正在构建的目标模型（而非假想中的模型）在真实问题上实验。阅读模型的执行 trace，理解它在哪里成功、在哪里失败。基于观察而非猜测来调优性能。

2. **按需分解，专门化 agent**。对于更复杂的任务，有时将任务分解并对每个方面应用专门的 agent 可以带来 headroom。但不要默认就分解——先验证单 agent 确实做不到。

3. **模型升级时重申 harness**。每当新模型发布，系统性地检查 harness 中的每个组件：剥离不再承载性能的组件（减少复杂度和成本），添加新组件以达成之前不可能的能力（利用新的模型能力）。

6.3 结构化反思

从本文的实践中可以提炼出几个层次的认知：

方法论层面：Harness 的每个组件都是一个可测试的假设（“模型不能独立完成 X”）。当模型进步时，假设需要重新验证。“假设-验证-迭代”构成了 harness 开发的核心循环。

架构层面：从 GAN 到 multi-agent 流水线的迁移展示了一个通用模式——将人类软件工程的最佳实践（代码审查、QA、sprint planning）**编码为 agent 角色**，而非试图让单个 agent 同时承担所有角色。角色分离使得每个 agent 的行为更易于独立调优。

工程实践层面：

- 评分标准本身就是有效的 prompt engineering——即使没有 evaluator 反馈，仅将评分标准写入 generator 的 prompt 就能提升 baseline
- 文件作为 agent 间通信媒介（file-based communication）是一种简单、可追溯且有效的设计
- Planner 应保持高层次以避免错误级联，sprint contract 负责弥合细节间隙
- QA agent 的校准是一个需要多轮人工审查的调优过程，不存在“一次配置，永久有效”的捷径
- Evaluator 的价值与任务难度和模型能力之间的关系是**动态的**

与上一篇 Harness Design 文章的关系

本文是 Anthropic 关于 harness design 系列的延续。上一篇文章（*Harness design for agentic coding*）建立了 initializer + coding agent + context reset 的基本框架。本文在此基础上：

- 引入了 GAN 启发的 generator-evaluator 分离
- 将主观设计质量纳入可评分框架
- 从 Sonnet 4.5 演进到 Opus 4.5/4.6，逐步简化架构
- 强调了 harness 组件的“保质期”概念——它们会随模型进步而过时

两篇文章共同构成了一个完整的 harness design 方法论。

6.4 拓展阅读

- Anthropic: *Building Effective Agents* — harness 设计的基础原则，“find the simplest solution possible”
- Anthropic: *Context Engineering* — 上下文窗口管理策略，context anxiety 的深入讨论
- Anthropic: *Harness Design for Agentic Coding* — 本文的前序文章，initializer + coding agent + context reset 架构

- **Claude Agent SDK** 文档 — 构建 multi-agent 系统的官方框架
- **Playwright MCP** — 让 agent 与真实浏览器交互的工具，evaluator 的核心能力来源
- Goodfellow et al., *Generative Adversarial Networks* (2014) — GAN 的原始论文，generator-discriminator 对抗训练的理论基础